

ASSIGNMENT 1: TECHNICAL ASSESSMENT REPORT

Module: SQL Injection Detection, Exploitation & Prepared Statement Remediation
Course Identifier: 2311CS040137-EH | Date: September 22, 2026

1. Executive Summary

This laboratory assessment focuses on in-depth analysis, controlled exploitation, and root-cause defense engineering for **SQL Injection (SQLi / CWE-89)** vulnerabilities. Using an isolated sandbox SQLite database and simulated DVWA attack surfaces, three distinct attack vectors were evaluated: Authentication Bypass, UNION-based confidential record extraction, and Numeric parameter manipulation. All discovered flaws were re-engineered with parameterized prepared statements and strict input validation, achieving 100% verified remediation.

2. Evaluated SQL Injection Attack Vectors

ID	Attack Vector	Injected Payload	Vulnerable Output / Impact	Remediation State
VULN-001	Authentication Bypass	admin' OR '1'='1' --	Bypassed password check; issued admin session token.	REMEDIATED
VULN-002	UNION Data Theft	' UNION SELECT 99, account_number, account_type, balance, 1, ssn_last4, 0 FROM accounts --	Leaked confidential account numbers, balances, and SSNs.	REMEDIATED
VULN-003	Numeric Tautology	1 OR 1=1	Dumped entire user table without authentication.	REMEDIATED

3. Code Architecture: Insecure vs. Remediated

Insecure Approach (Raw String Interpolation - CWE-89):

```
# VULNERABLE: Direct string interpolation alters query structure
raw_query = f"SELECT * FROM users WHERE username = '{username}' AND password_hash = '{pw_hash}'"
cursor.execute(raw_query)
```

Hardened Approach (Prepared Statements & Regex Whitelisting):

```
# SECURE: Input validation + Parameterized prepared statement placeholders
if not re.match(r'^[a-zA-Z0-9_\-]{3,30}$', username):
    return {"status": "validation_error", "error": "Illegal username characters"}
query = "SELECT id, username, email FROM users WHERE username = ? AND password_hash = ?"
cursor.execute(query, (username, pw_hash))
```

4. Automated Regression & Verification Results

Automated regression test harnesses (test_sql_i_vulnerable.py & test_sql_i_secure.py) were executed to validate the efficacy of defensive controls:

Test Harness	Tests Executed	Exploit Success (Vuln)	Breach Rate (Secured)	Outcome
test_sql_i_vulnerable.py	6 Vectors	100% (6/6 Succeeded)	N/A	VULN CONFIRMED

test_sqli_secure.py	6 Vectors	0% (0/6 Succeeded)	0.00% Breaches	100% SECURE
---------------------	-----------	--------------------	----------------	-------------

5. Evidence Inventory & Cryptographic Signatures

Evidence ID	Target Vector	Log Artifact	SHA-256 Checksum
EV-SQLI-001	Auth Bypass PoC	EV-SQLI-001_auth_bypass.md	87f7b1b7b4a7...
EV-SQLI-002	UNION Extraction PoC	EV-SQLI-002_union_extract.md	2cc64a5d0f04...
EV-SQLI-003	Numeric Tautology PoC	EV-SQLI-003_blind_boolean.md	c5a349d3e0b7...